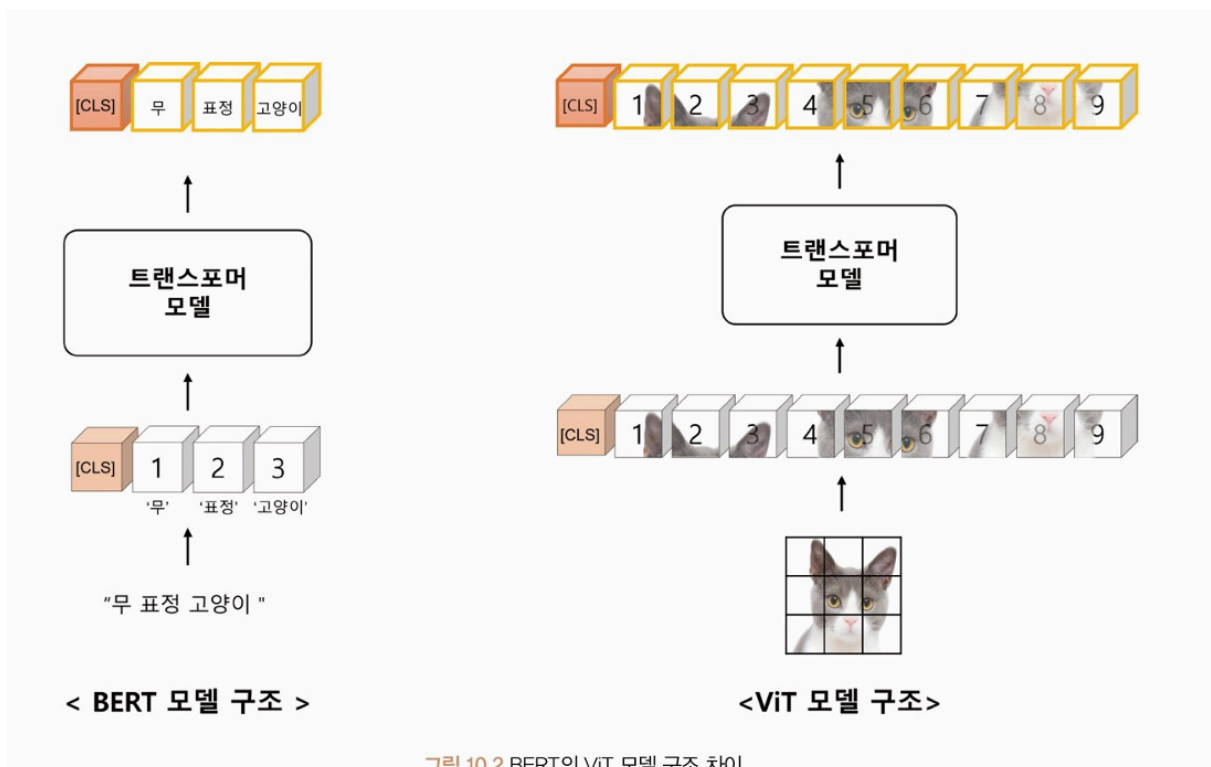


10장 비전 트랜스포머

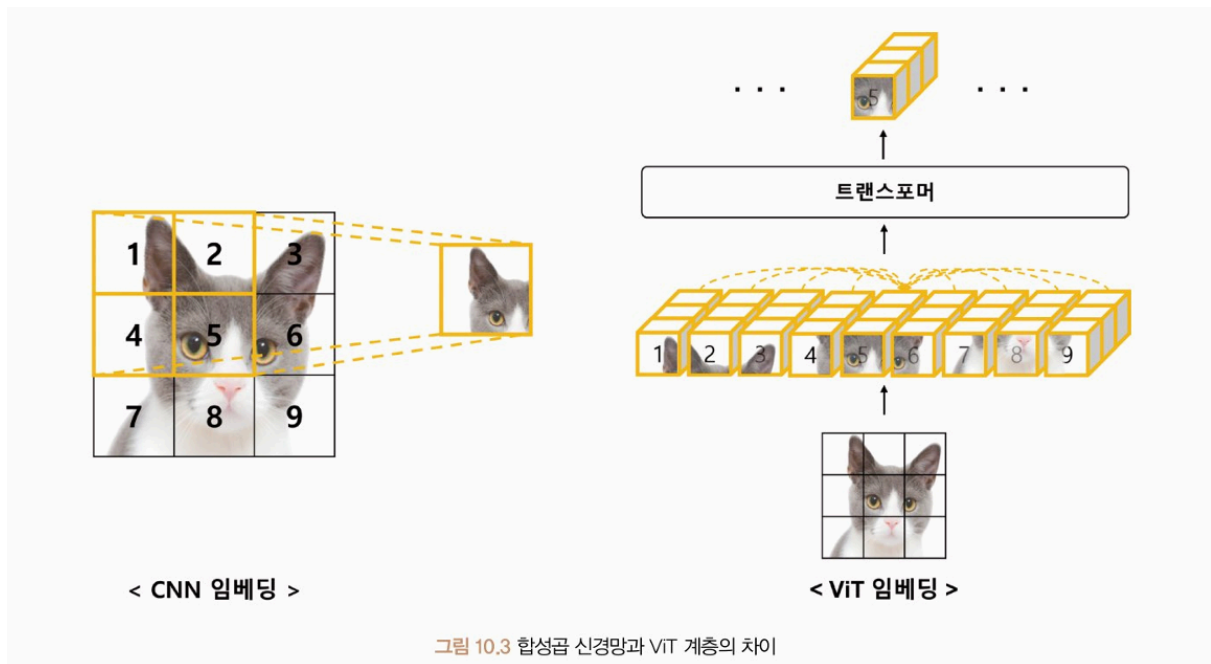
ViT (Vision Transformer)

트랜스포머 구조 자체를 컴퓨터비전 분야에 적용한 첫 번째 연구



BERT 모델(자연어 처리에 널리 사용되는 모델) - 문장보다 작은 단위의 토큰들이 순차적으로 입력 <> ViT 모델 - 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 순차적으로 입력됨

합성곱 신경망 vs ViT



• 합성곱 신경망(CNN) 임베딩 :

- 이미지 패치 중 **일부만** 선택하여 학습하며, 이를 통해 이미지 전체의 특징을 추출
- 픽셀 단위로 처리
- 이미지의 공간적인 위치 정보 고려

• ViT 임베딩 :

- 이미지를 작은 패치들로 나눠 패치 간의 상관관계를 학습,
셀프 어텐션 방법을 사용해 **모든 이미지 패치-**가 서로에게 주는 영향을 고려해 이미지의 전체 특징을 추출
- 패치 단위로 이미지를 처리 → 작은 모델로도 높은 성능
- 입력 이미지의 크기 고정 → 크기가 다른 이미지를 처리하려면 전처리 필요
- 패치 간의 상대적인 위치 정보만 고려 → 이미지 변환에 취약

⇒ CNN은 수용 영역(Receptive Field) 이 좁아 전체 이미지 정보를 표현하는데 **수많은 계층**이 필요 반면 ViT 는 어텐션 거리(Attention Distance) 를 계산하여 **오직 한 개의 ViT 레이어**로 전체 이미지 정보를 쉽게 표현할 수 있다.

- * 수용 영역 (Receptive Field)
- * 어텐션 거리(Attention Distance)

ViT의 귀납적 편향(Inductive Bias)

딥러닝 모델에서 일반화 성능 향상을 위한 모델의 가정(Assumption)

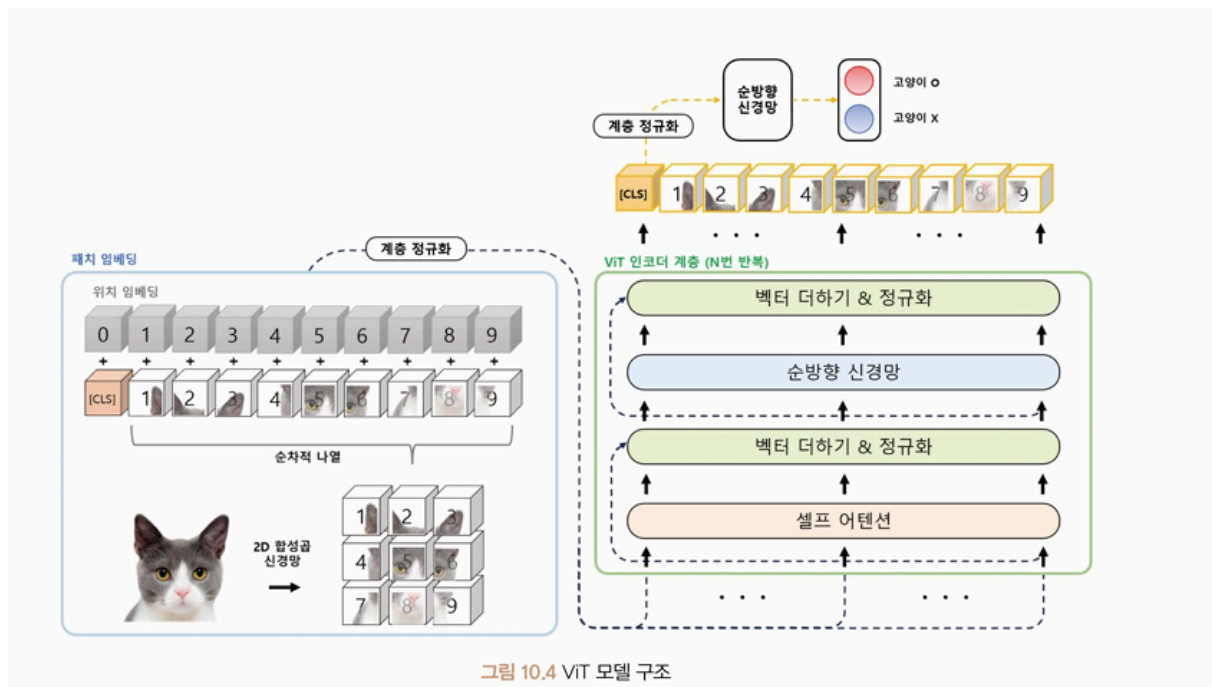
- 이미지 데이터 : 공간적 관계를 잘 표현하므로 지역적(local) 편향을 가진 합성곱 신경망 모델 활용

- 시계열 데이터 : 시간적 관계를 잘 표현하므로 시퀀셜(sequential) 편향을 가진 순환 신경망 모델

이러한 관계 편향이 강할수록 다양한 관계를 표현하기 어려워 귀납적 편향이 약한 모델이 선호됨

⇒ ViT 모델은 입력 데이터의 다양한 쿼리(Query), 키(Key), 값(Value)의 임베딩 형태로 일반화된 관계를 학습하기 때문에 **귀납적 편향이 거의 없고** 대용량 데이터에 대해서 이미지 특징들의 관계를 잘 학습함.

Vit 모델



- **패치 임베딩(Patch Embedding)** : 입력 이미지를 트랜스포머 구조에 맞게 일정한 크기의 패치로 나눈 다음 각 패치를 벡터 형태로 변환
- **인코더(Encoder)** : 각 패치와의 관계 학습

패치 임베딩(Patch Embedding)

입력 이미지를 작은 패치로 분할하는 과정

- 이미지 크기를 맞추는 전처리 (정방향 크기로 조절)
- 전체 이미지를 패치 크기로 분할

수식 10.1 패치 계산 과정

$$patch\ size = \frac{image\ size - kernel\ size}{stride} = \frac{9 - 3}{3} + 1 = 3$$

- **시퀀셜 배열 만들기**

왼쪽에서 오른쪽-위에서 아래 순서로 배열.

가장 왼쪽에 분류 토큰(Special Classification Token) 을 추가해 전체 이미지를 대표하는 벡터로 특정 문제를 예측하는 데 사용된다.

- **위치 임베딩(Position Embedding)**

인접한 패치 간의 관계 학습

패치의 위치 정보를 임베딩 벡터로 변환하고 기존 이미지 벡터들과 더함

- **계층 정규화**

인코더 계층

- 입력 : 패치 임베딩 계층을 거친 벡터들
- N 개의 인코더 레이어를 반복적으로 적용한 후 마지막 레이어에서 분류 토큰의 특징 벡터 추출
 - ⇒ 다양한 이미지 분류 및 검색 문제를 해결하는 데 사용
 - ⇒ 전체 시퀀셜 산출물 (Sequential Output) 을 모두 사용하는 것이 아니라 전체 이미지의 특징을 잘 표현하는 분류 토큰 벡터만 사용
- 분류 토큰 벡터를 순방향 신경망 또는 완전 연결 계층에 연결하고 실제 분류를 수행한다.

실습(ViT)

- FashionMNIST 데이터를 불러오고 균등 샘플링
- 이미지 데이터 전처리
 - 이미지 프로세서 클래스(AutoImageProcessor) : 사전 학습된 ViT 모델을 활용해 전처리를 진행
 - PIL.Image 형식이 아닌 Tensor 형식을 사용하므로 변환 모듈을 통해 이미지를 변환
 - 크기 조정 (transforms.Resize)

- 사용자 정의 함수 클래스(transforms.Lambda) 단일 채널을 복제해 다중 채널 이미지로 변환
 - 정규화 (transforms.Normalize)

- 데이터로더 적용

Vit 모델은 Tensor 형식이 아닌 딕셔너리 형식

```
{"pixel_values": pixel_values, "labels": labels}
```

형태를 입력으로 사용

- 집합 함수 (collate_fn) 을 적용해 미니 배치의 샘플 목록을 병합
- 사전 학습된 ViT 모델 불러오기
- 모델 학습
 - 패치 임베딩 구조 확인
트랜스포머 모델과 유사하지만 입력 데이터를 일정한 크기의 패치로 분할한 후 패치마다 특성을 추출하는 과정을 추가로 수행
 - 패치 임베딩 함수(model.vit.embedding.patch_embeddings) 는 16x16 커널과 16 간격의 합성곱 연산을 수행. 224x224 크기의 이미지에 합성곱 연산을 수행한다면 14x14 의 텐서가 생성된다. 이 텐서를 일렬로 나열해 196개의 벡터 생성
 - 임베딩 함수 (model.vit.embedding) : 이 함수 앞에 [4,7,768] 크기의 [CLS] 토큰을 붙혀 197개의 벡터를 생성.
 - 하이퍼파라미터 설정
 - 학습 매개변수 클래스(TrainingArguments) : 모델 학습에 필요한 다양한 인자들을 저장하고관리할 수 있는 클래스.
 - 현재 예제에서 사용된 매개변수들

표 10.4 학습 매개변수 옵션식(training arguments)

매개변수	의미
output_dir	체크포인트 저장 경로
save_strategy	체크포인트 저장 간격 설정 - no: 저장 안 함 - steps: 스텝마다 - epoch: 에폭마다
save_steps	save_strategy= "steps"일 때 스텝 간격 설정
save_total_limit	체크포인트 최대 저장 개수
evaluation_strategy	체크포인트 평가 간격 설정 - no: 저장 안 함 - steps: 스텝마다 - epoch: 에폭마다

매개변수	의미
learning_rate	초기 학습률
per_device_train_batch_size	학습 배치 크기
per_device_eval_batch_size	평가 배치 크기
num_train_epochs	학습 반복 수
weight_decay	가중치 감쇠
load_best_model_at_end	모델 불러오기 시 최상의 모델 선택 여부
metric_for_best_model	최상의 모델 선정 기준이 되는 평가 방식 설정 - accuracy: 정확도 - precision: 정밀도 - f1: F1 점수 - mae: 평균 절대 오차 - mse: 평균 제곱 오차 - rmse: 평균 제곱근 편차
logging_dir	로그 저장 폴더
logging_steps	로그 출력 간격
seed	학습 시작 시 설정되는 무작위 시드
fp16 ⁵	Mixed precision 사용 여부

- 최종 학습 코드
 - 평가 함수
 - 매크로 평균 F1 점수 (Macro Average F1-Score)
 - 최종 성능 평가 코드
- 혼동 행렬(Confusion Matrix) 를 활용해 예측 결과를 확인해 보기

Swin Transformer

기존 트랜스포머 모델의 문제점

- 입력 이미지 크기에 대한 2차(Quadratic) 계산 복잡도 $\Rightarrow$ 고해상도 이미지를 처리하기 어려움
- 고정적인 패치 크기로 인해 세밀한 예측을 필요로 하는 작업(의미론적 분할, Semantic Segmentation) 과 같은 작업에 적합하지 않음

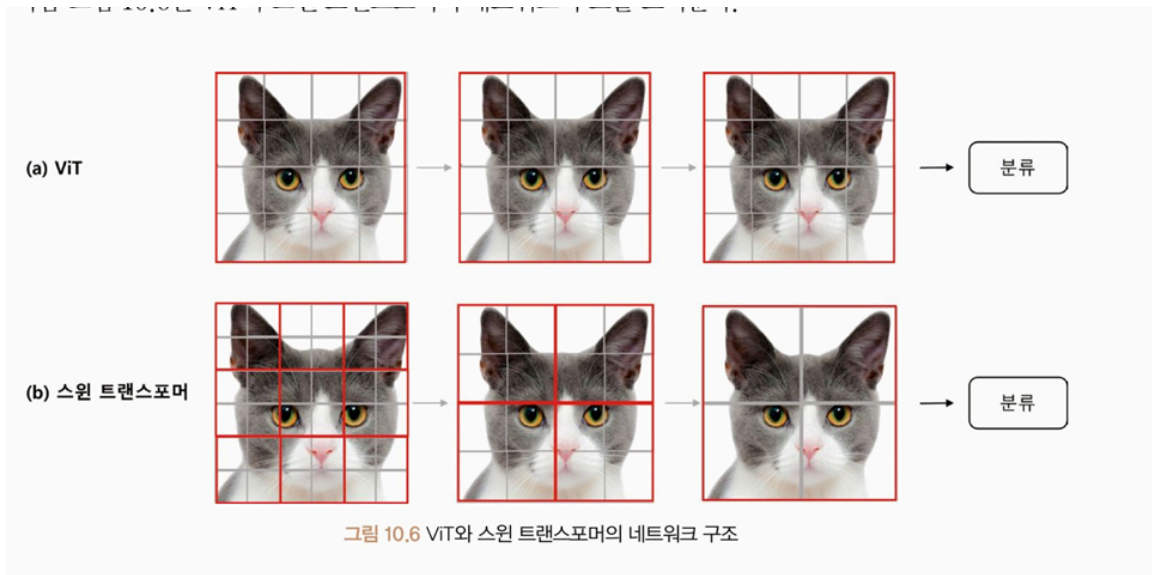
$\Rightarrow$ Swin Transformer 는 계층적(hierarchy) 특징 맵을 구성해 **1차(Linear) 계산 복잡도**를 갖고, 시프트 윈도우(Shifted Window) 라는 기술을 이용해 입력 이미지를 일정한 크기의 패치로 분할하고 이 패치를 쌓아 윈도우를 구성, 구성된 윈도우 영역만 셀프 어텐션을 계산

ViT 와 스윈 트랜스포머 차이

- ViT 모델
 - 획일적인 패치 크기, 위치에 대한 제약 $\Rightarrow$ 다양한 이미지크 크기와 종횡비를 가진 데이터 세트 적용 힘들
 - 패치 단위로 입력을 받아 이미지의 공간 정보가 완전히 보존되지 않을 수 있음
 - 패치 수가 증가하면 계산량이 증가
- 스윈 트랜스포머
 - 로컬 윈도우 (Local Window) 를 활용해 물체의 크기나 해상도를 계층적으로 학습.
 - 시프트 윈도우 (Shifted Window) : 로컬 윈도우를 이동

$\Rightarrow$ ViT 보다 높은 정확도, 더 적은 매개변수, 더 넓은 범위의 이미지 크기/종횡비 처리 가능

 - 로컬 윈도우를 통해 계층마다 어텐션이 되는 패치의 크기와 개수를 계층적으로 다양하게 적용



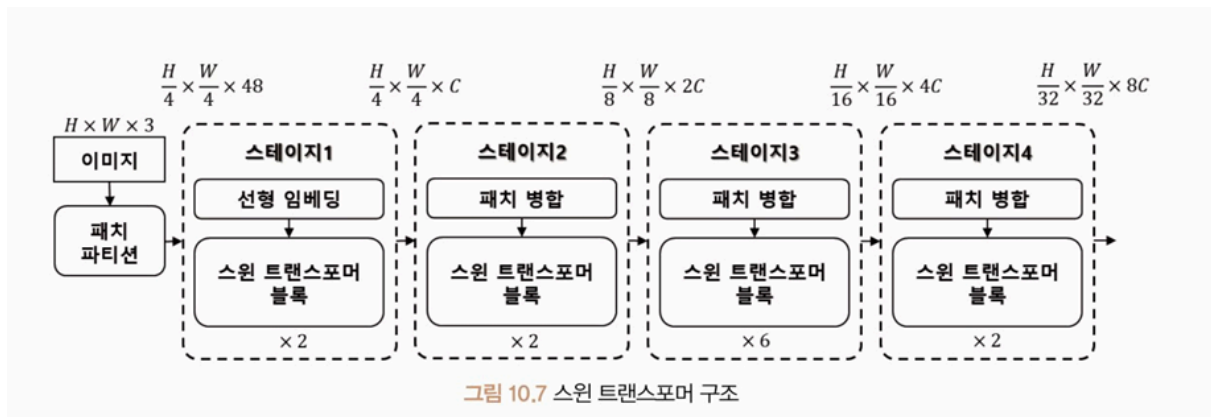
• ViT 와 스윈 트랜스포머 비교

표 10.4 ViT와 스윈 트랜스포머 비교

	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도우 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

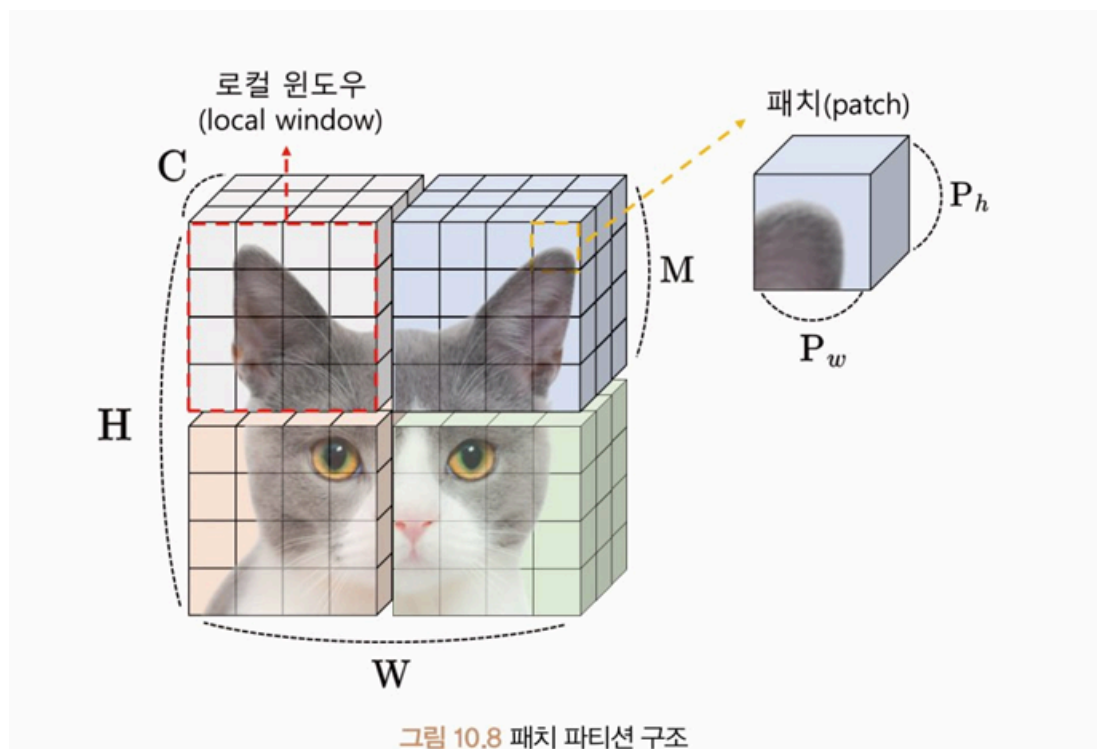
스윈 트랜스포머 모델 구조

이미지를 패치 파티션으로 구분한 후 선형 임베딩(Linear Embedding), 스윈 트랜스포머 블록(Swin Transformer Block), 패치 병합(Patch Merging) 연산을 반복적으로 수행



1. 패치 파티션/패치 병합

- 패치 파티션(Patch Partition) : 입력 이미지를 작은 사각형 패치로 분할해 처리하는 방식.
분할된 패치는 트랜스포머 계층의 입력으로 사용됨.

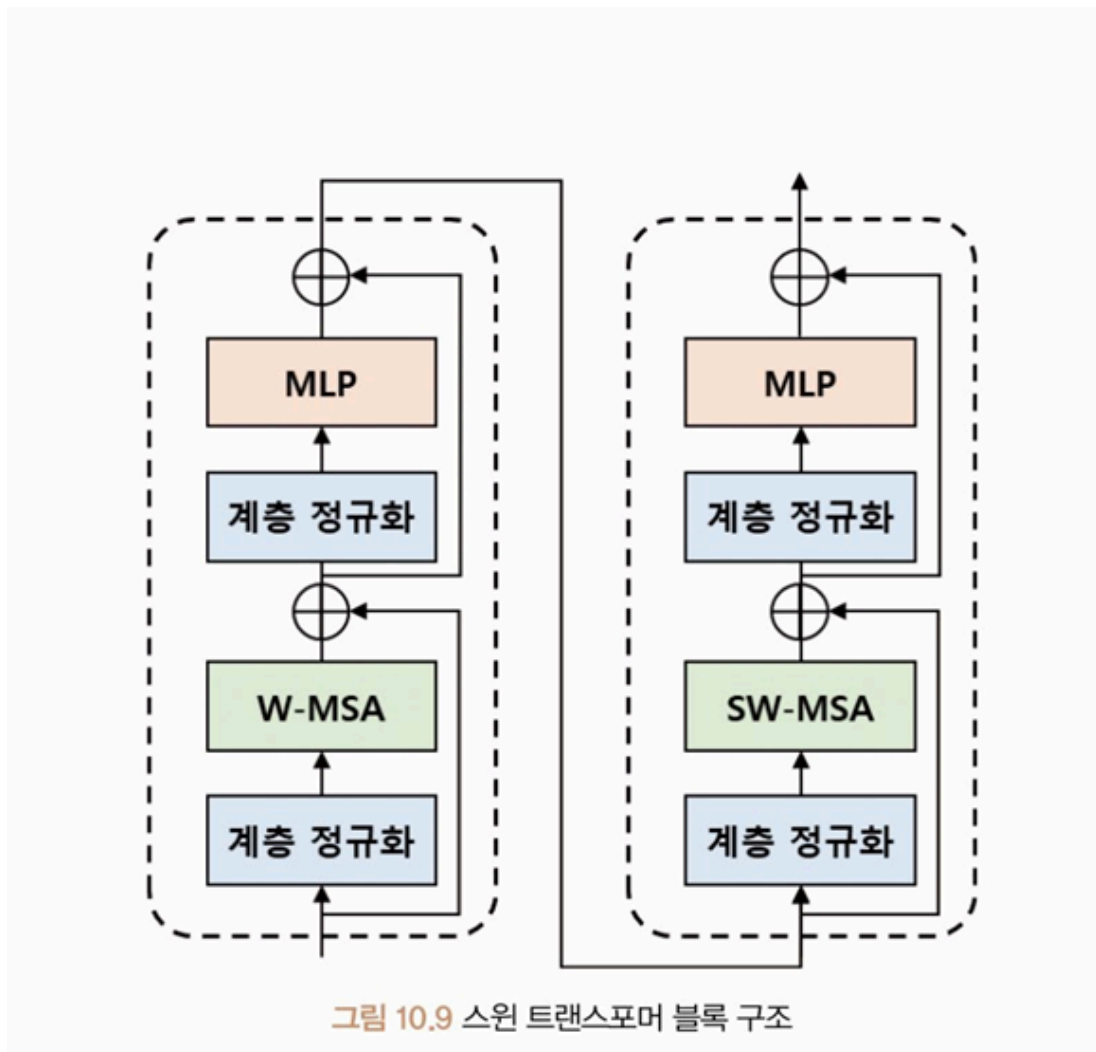


일반적인 $[C, H, W]$ 의 구조에서 패치 파티션을 통해 로컬 윈도우(빨간 점선) 를 추가한다.

- 패치 병합 (Patch Merging) : 인접한 패치들의 정보를 저차원으로 축소하는 과정
모델 매개변수의 수를 줄이기 위해 $[C, H, W] \Rightarrow [2C, H/2, W/2]$ 로 재정렬한 후 $2C$ 의 차원을 원래 차원(저차원)으로 임베딩

2. 스윈 트랜스포머 블록

- 패치 파티션 이후 네 개의 스테이지 안에서 반복 적용, 선형 임베딩 또는 패치 병합 이후 수행된다.



- W-MSA
로컬 윈도우를 사용하는 멀티 셀프 어텐션, 입력 특징 맵을 중첩되지 않게 나눈 다음 각 윈도우에서 독립적으로 셀프 어텐션 수행

<장점>

- 일정한 크기의 윈도우 영역 설정 ⇒ 해당 영역 내 픽셀 간의 어텐션 계산
- 이미지 내의 전체적인 어텐션 계산, 특정 위치에 대한 정보 추출
- 각 윈도우 내 지역적인 특징 분석 가능
- 전체 입력에 대한 셀프 어텐션 비용을 1차 계산 복잡도로 줄임 ⇒ 연산량 감소 !!

<단점>

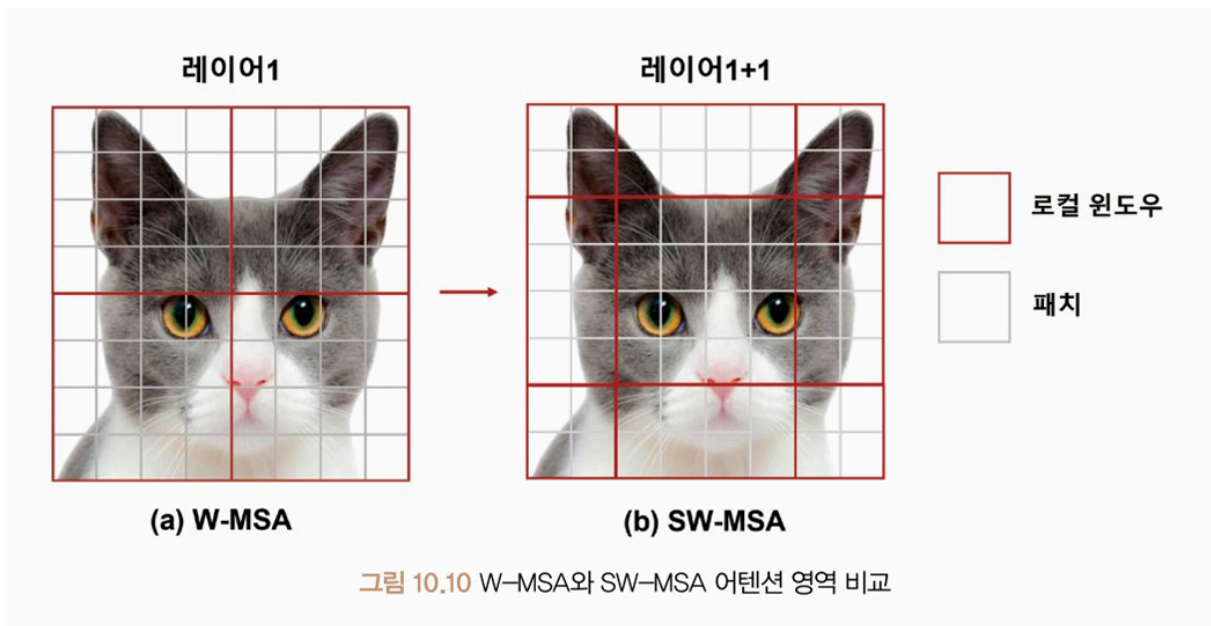
- 원도 내부의 이미지 패치 영역에만 셀프 어텐션을 수행 $\Rightarrow$ 원도 간의 관계성 정보 추출 불가능

- SW-MSA

W-MSA 의 단점에서 착안, 원도 사이의 연결성을 구축해 원도 간의 관계성 정보 추출. 원도를 가로 또는 세로 방향으로 이동한 후 인접한 원도 간의 셀프 어텐션을 계산하는 방식

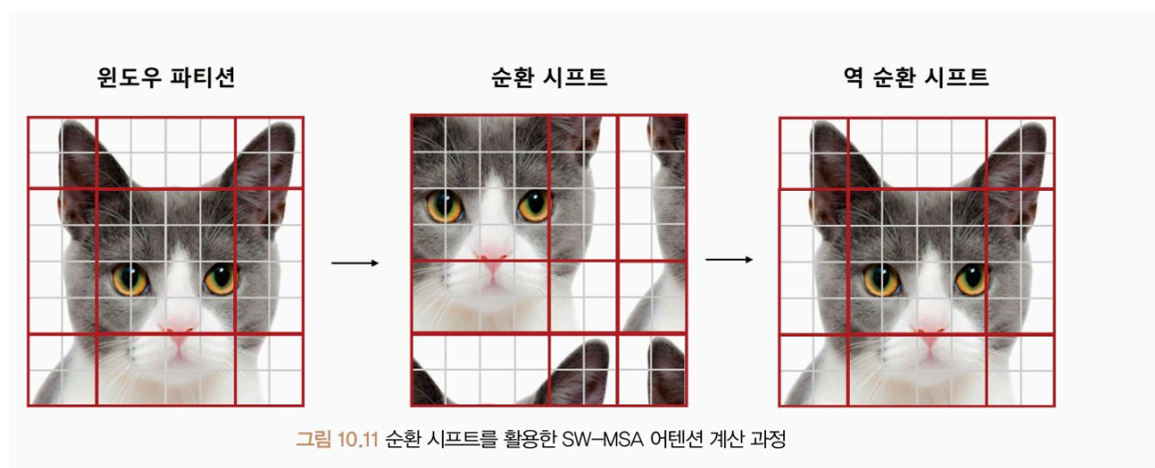
<단점>

- 사용된 로컬 원도 개수가 더 많아 더 많은 셀프 어텐션 연산을 요구



- 순환 시프트 (Cyclic Shift)

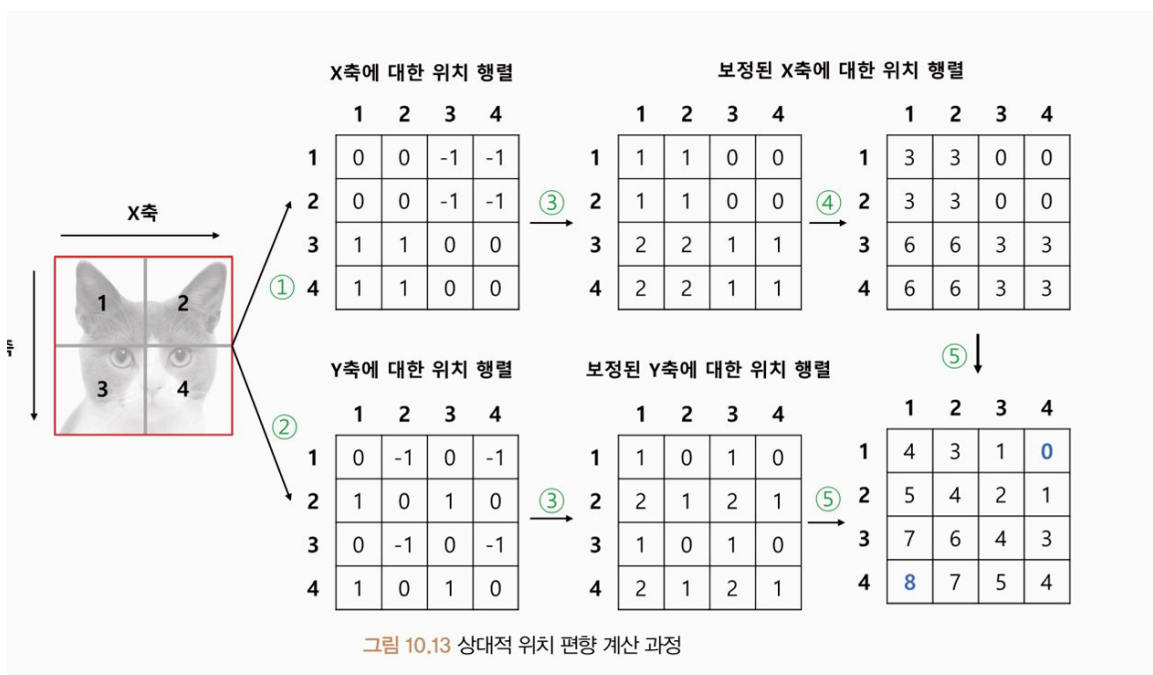
로컬 원도가 이동할 때 이동한 위치의 정보를 이전 위치에서 가져오는 것. 로컬 원도 사이즈 M 보다 작은 $M/2$ 만큼 로컬 원도를 이동시킴.



- 어텐션 마스크 (Attention Mask)
이동된 위치에서 연산을 수행하지 않도록 방지하는 역할.
- 역순환 시프트 (Reverse Cyclic Shift)
원래 로컬 윈도우 구조로 복원

⇒ 4개의 윈도우 파티션에 대한 4개의 셀프 어텐션 값만으로 모든 로컬 윈도우 간의 셀프 어텐션 값을 계산할 수 있음 .

- 상대적 위치 편향(Relative Position Bias)
로컬 윈도우 안에 있는 패치 간의 상대적인 거리를 임베딩하는 목적



- 코드로 구현해보기

#상대적 위치 편향 계산

```
import torch
```

```
window_size = 2
```

```
coords_h = torch.arange(window_size)
```

```
coords_w = torch.arange(window_size)
```

```
coords = torch.stack(torch.meshgrid([coords_h, coords_w], indexing="ij"))
```

```
coords_flatten = torch.flatten(coords, 1)
```

```
relative_coords = coords_flatten[:, :, None] - coords_flatten[:, None, :]
```

```

print(relative_coords)
print(relative_coords.shape)import torch

window_size = 2
coords_h = torch.arange(window_size)
coords_w = torch.arange(window_size)
coords = torch.stack(torch.meshgrid([coords_h, coords_w], indexing="ij"))
coords_flatten = torch.flatten(coords,1)
relative_coords = coords_flatten[:, :, None] - coords_flatten[:, None, :]
print(relative_coords)
print(relative_coords.shape)

```

#X,Y 축에 대한 위치 행렬

```

x_coords = relative_coords[0,:,:]
y_coords = relative_coords[1,:,:]

x_coords += window_size - 1
y_coords += window_size - 1
x_coords *= window_size - 1

print(f"X축에 대한 위치 행렬 : \n{x_coords}\n")
print(f"Y축에 대한 위치 행렬 : \n{y_coords}\n")

relative_position_index = x_coords + y_coords
print(f"X,Y 축에 대한 위치 행렬 : \n{relative_position_index}\n")

```

#X,Y 축에 대한 상대적 위치 좌표 변환

```

num_heads=1
relative_position_bias_table = torch.Tensor(
    torch.zeros((2*window_size - 1) * (2*window_size - 1), num_heads)
)

relative_position_bias = relative_position_bias_table[relative_position_index]
relative_position_bias = relative_position_bias.view(
    window_size * window_size, window_size * window_size, -1
)

```

```
)  
print(relative_position_bias.shape)
```

결과

```
⇒ torch.Size([4, 4, 1])
```

- 수식

수식 10.2 상대적 위치 편향을 고려한 셀프 어텐션

$$Attention(Q, K, V) = SoftMax(QK^T / \sqrt{d} + B)V$$

실습(스윈 트랜스포머)

- 허깅 페이스 라이브러리에서 제공하는 사전 학습 모델 사용
 - ImageNet-1k 데이터세트
 - 224×224 이미지 크기
 - 4×4 이미지 패치
 - 7×7 로컬 윈도우
- 사전 학습된 스윈 트랜스포머 모델
형태

```

swin
├── embeddings
│   ├── patch_embeddings
│   │   └── projection Conv2d(3, 96, kernel_size=(4, 4), stride=(4, 4))
│   ├── norm
│   ├── dropout
│   └── encoder
│       ├── layers
│       │   ├── 0
│       │   ├── 1
│       │   ├── 2
│       │   └── 3
│       ├── layernorm
│       └── pooler
└── classifier

```

- 패치 임베딩 모듈 확인
- SwinLayer 구조 확인하기

```

SwinLayer(
  (layernorm_before): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
  (attention): SwinAttention(
    (self): SwinSelfAttention(
      (query): Linear(in_features=96, out_features=96, bias=True)
      (key): Linear(in_features=96, out_features=96, bias=True)
      (value): Linear(in_features=96, out_features=96, bias=True)
      (dropout): Dropout(p=0.0, inplace=False)
    )
    (output): SwinSelfOutput(
      (dense): Linear(in_features=96, out_features=96, bias=True)
      (dropout): Dropout(p=0.0, inplace=False)
    )
  )
  (drop_path): Identity()
  (layernorm_after): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
  (intermediate): SwinIntermediate(
    (dense): Linear(in_features=96, out_features=384, bias=True)
    (intermediate_act_fn): GELUActivation()
  )
  (output): SwinOutput(
    (dense): Linear(in_features=384, out_features=96, bias=True)
    (dropout): Dropout(p=0.0, inplace=False)
  )
)

```

기본적인 빌딩 블록이 되는 계층으로, 스윈 트랜스포머 블록을 의미.

<순서>



계층 정규화 → W-MSA/SW-MSA → 계층 정규화 → MLP

- SwinIntermediate / SwinOutput : MLP 계층

두 개의 연속된 선형 변형 계층과 GELU(Gaussian Error Linear Unit) 활성화 함수로 구성

★ GELU : ReLU 와 유사한 형태를 가지며, 가우시안 분포에 대한 누적 분포 함수

⇒ 입력 특징맵을 더 깊은 차원으로 매핑하고 ReLU 를 사용했을 때보다 더 안정적으로 학습 가능

- W-MSA, SW-MSA 모듈



패치 임베딩 차원 : `torch.Size([32, 3136, 96])`

W-MSA 결과 차원 : `torch.Size([32, 3136, 96])`

SW-MSA 결과 차원 : `torch.Size([32, 3136, 96])`

W-MSA 모듈, SW-MSA 모듈의 출력 차원이 동일한 하이퍼파라미터로 구성된 것을 확인할 수 있다.

- 패치 병합



```
patch_merge 모듈 : SwinPatchMerging(
  (reduction): Linear(in_features=384, out_features=192, bias=False)
  (norm): LayerNorm((384,), eps=1e-05, elementwise_affine=True)
)
patch_merge 결과 차원 : torch.Size([32, 784, 192])
```

- reduction

- 계층 정규화

⇒ 3136(56×56) 개 패치가 784(28×28) 개로 분할

- 분할된 패치는 4개로 나뉘어 (N, 28, 28, 96) 의 텐서 생성

- 이 텐서를 채널에 넣어 병합하면 $96 \times 4 = 384$

⇒ 채널이 줄어들지 않으므로 선형 변환계층을 통해 채널의 개수 절반으로 줄임 (384→192)

- patch_merge 결과 차원 [N, 784, 192]

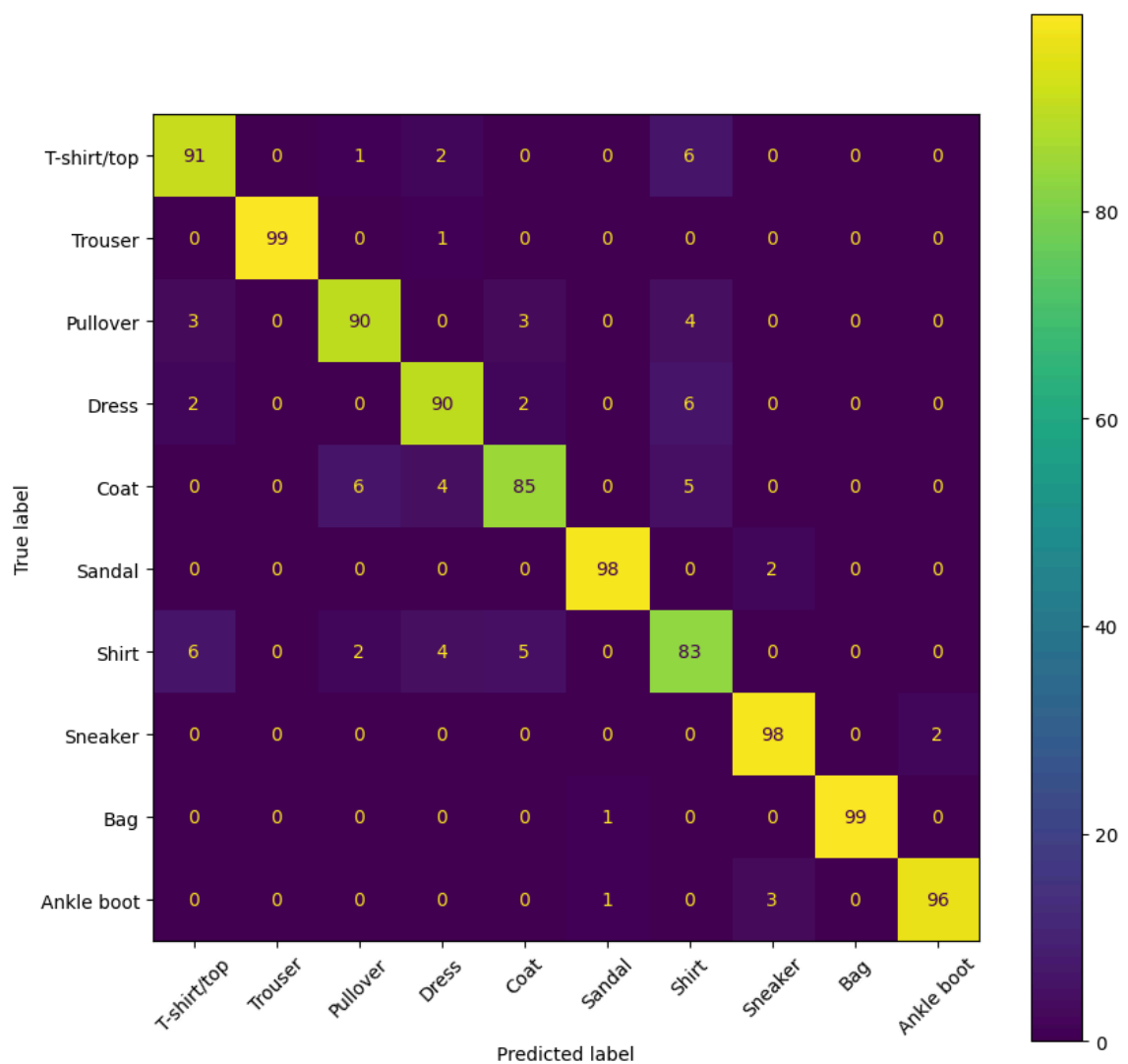
- 4개의 스테이지를 모두 통과할 때마다 패치 크기와 수는 많아지며 토큰의 차원은 두 배씩 증가

⇒ 최종 크기 $[N, 49(7 \times 7), 768]$

- 스윈 트랜스포머 모델 학습

Epoch	Training Loss	Validation Loss	F1
1	0.346800	0.300104	0.897595
2	0.250200	0.254696	0.910317
3	0.221400	0.225194	0.929172

- 성능 평가



CvT (Convolutional Vision Transformer)

합성곱 신경망 구조에서 활용하는 계층형 구조(Hierarchical Architecture) 를 ViT 에 적용한 모델

- Vit 의 셀프 어텐션 : 모든 패치에 대해 동일한 크기와 간격을 사용하므로 이미지의 물체 크기나 해상도를 계층적으로 학습하기 어려움
- 스윈 트랜스포머 : 트랜스포머 블록마다 다른 로컬 윈도우 크기를 사용해 이미지의 계층적 특징 학습
- CvT : 합성곱 신경망에서 사용하는 계층적 구조를 ViT 에 적용해 이미지의 지역 특징 (Local Feature) 과 전역 특징(Global Feature) 를 모두 활용해 적은 수의 매개변수로 높은 성능을 이끌어냄
 - ⇒ 합성곱 계층의 지역 특징을 추출하고 ViT 의 셀프 어텐션 계층으로 전역 특징을 결합해 이미지 처리

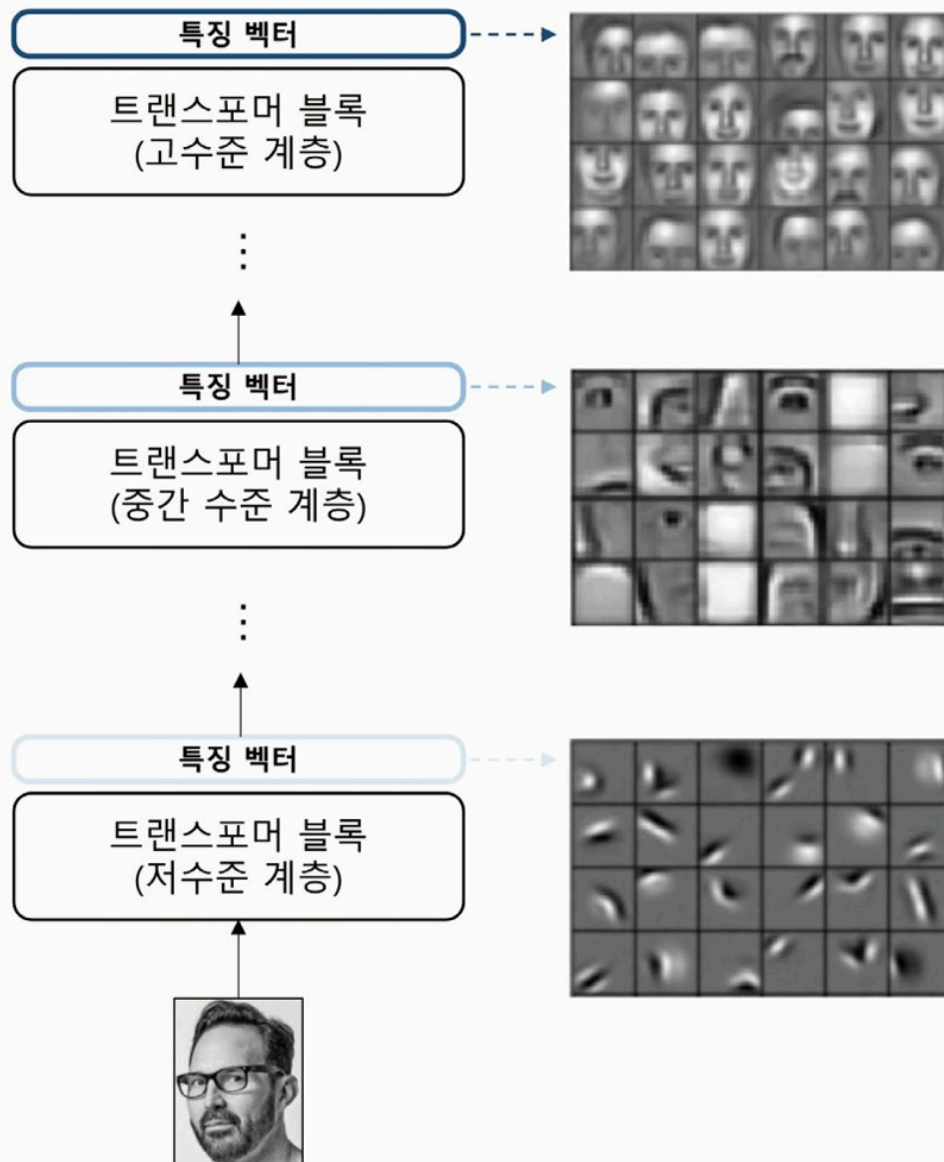


그림 10.14 CvT 모델의 계층적 특징 구조 예시

⇒ 토큰에 대한 위치 임베딩을 적용하지 않아도 모델의 성능 저하 없이 다양한 해상도를 가진 이미지를 처리할 수 있다.

표 10.4 ViT와 CvT 모델 비교

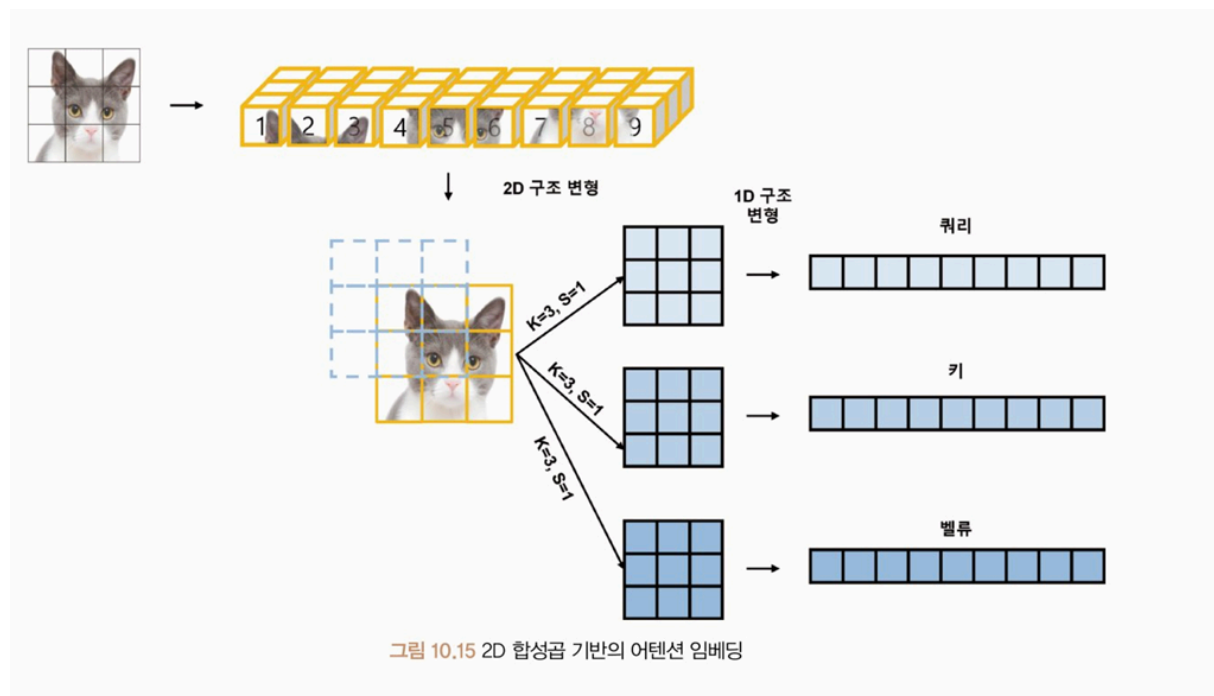
모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

합성곱 토큰 임베딩(Convolutional Token Embedding)

2D 로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수행하는 임베딩.

⇒ ViT 에 계층적인 합성곱 신경망의 성질을 추가

⇒ 이미지 패치를 2D 합성곱 임베딩을 사용해 쿼리, 키, 값 임베딩으로 변환, 이미지의 2D 구조를 보존할 수 있도록 함.



각각의 패치를 쿼리/키 값에 대해 합성곱 연산을 적용하고 1차원 구조로 재배열해 기존 셀프 어텐션 방법으로 학습하는 방식

어텐션에 대한 합성곱 임베딩(Convolutional Projection for Attention)

2D 로 재구성된 토큰 맵에서 분리 가능한 합성곱 연산을 적용해 성능 저하 및 계산 복잡도를 낮추는 연산.

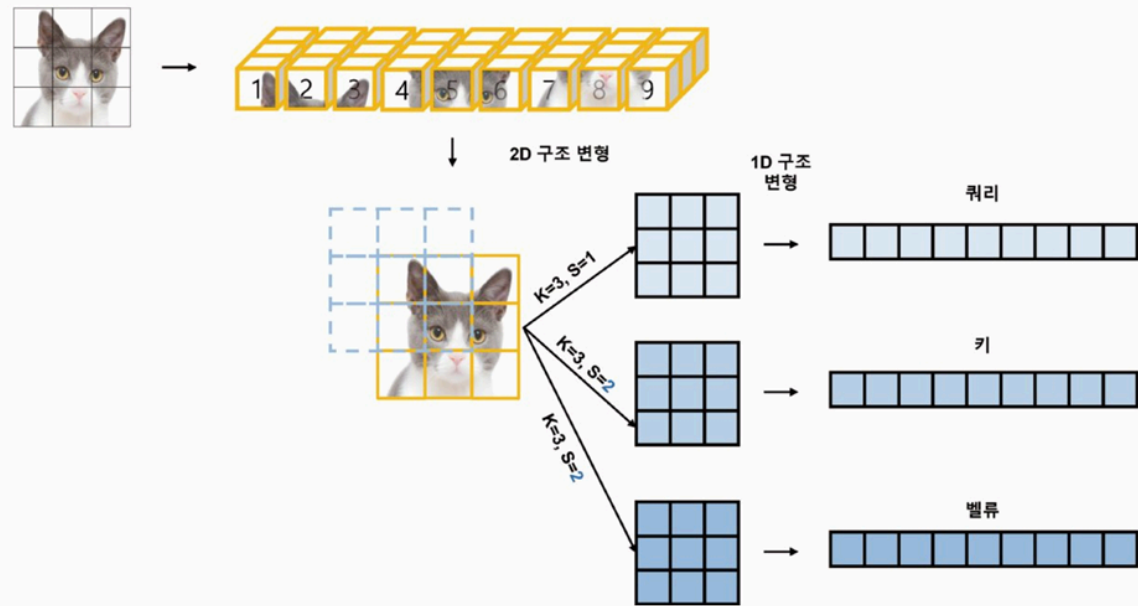


그림 10.16 2D 합성곱 기반의 축소된 어텐션 임베딩

키, 벨류 에 스트라이드 2 를 적용해 4개의 이미지 패치 생성

★ 길이가 다른 쿼리(9), 키(4), 값(4) 을 사용하더라도 곱셈의 사이즈는 9x1 로 동일하다.

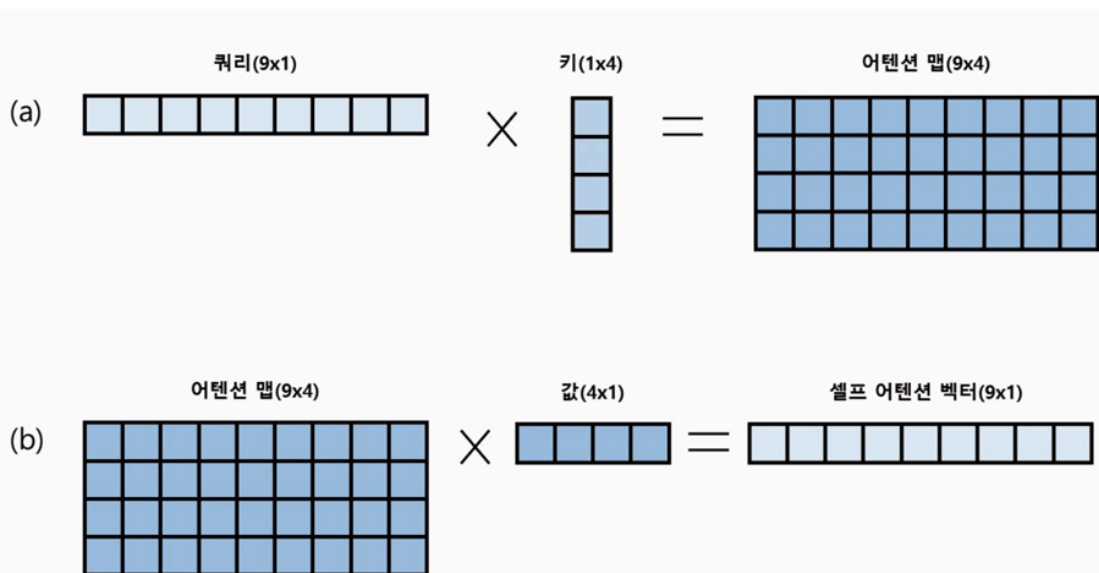


그림 10.17 축소된 셀프 어텐션 임베딩(열 × 행) 예시

- 어텐션 맵(Attention Map) : 키 패치 임베딩에 대한 중요도를 의미

트랜스포머 계층이 깊어질수록 패치 간의 관계를 학습하는 길이를 축소시키는 대신, **벡터의 차원을 증가시켜** 모델의 매개변수 수를 대폭 감소시킴

모델 실습(CvT)

- CvT 모델 이미지 데이터 전처리
 - `shortest_edge` 키 사용해서 전처리
: 이미지의 너비나 높이 중 더 작은 값.
 - `microsoft/cvt-21` 모델은 모든 입력 이미지의 크기를 가장 짧은 이미지 길이로 정규화해 사용한다.
- 사전 학습된 CvT 모델

```
cvt
├─ encoder
│   └─ stages
│       ├── 0
│       ├── 1
│       └── 2
├─ layernorm
└─ classifier
```

- CvT 모델의 스테이지 구조
 - `CvtEmbeddings`
 - 입력 이미지에 대한 2D 합성곱 연산과 계층 정규화로 이루어짐
입력 이미지 → (7x7 커널, 4 스트라이드, 2 패딩) 연산 진행
 - 이미지를 패치 단위로 잘라 임베딩하는 역할
 $224 \times 224 \rightarrow 64, 56 \times 56$
 - `CvLayer`
 - 매개변수가 축소된 어텐션 임베딩을 계산하기 위해 키와 값 임베딩의 간격을 1보다 큰 2로 설정해 차원을 축소
 - 3136 (56×56) 개의 이미지 패치에 대한 셀프 어텐션을 수행
- 셀프 어텐션 적용

```
⇒ torch.Size([32, 3, 224, 224])
   torch.Size([32, 64, 56, 56])
   torch.Size([32, 3136, 64])
   torch.Size([32, 3136, 64])
```

- 합성곱 계층을 통해 계산된 패치 임베딩을 가로, 세로 크기로 재정렬해 CvtLayer에 입력
- 모듈의 입력 차원과 출력 차원이 동일한 것 확인 가능
- CvT 모델 학습

Epoch	Training Loss	Validation Loss	F1
1	0.805000	0.329982	0.886639
2	0.684400	0.275579	0.907541
3	0.669700	0.265388	0.911864

- 평가

